CS-229 - review

Greg Ver Steeg

- Final logistics
- Go over Quizzes
- Final organization / topic review

Also course evals: https://ieval.ucr.edu
I'll do one email reminder

Final logistics

- ProctorU, Take online – I don’t know how this works
  - You will need to install the ProctorU Extension for Google Chrome or use their designated secure browser (Guardian)
  - (Honestly this looks quite tedious – let me know your experience, I may not use it again)
- Can use notes / scratch paper / calculator
- Test content: variations on Canvas quiz questions from quizzes and homework

Final structure

1-11

- Quick / conceptual questions

12-16

- Homework questions (simple variations of coding quiz)

17-20

- Calculation questions

Topics

- ML Basics:
  - Softmax, MLP, convolution
- Transformer basics:
  - BERT/GPT, self-attention, temperature, tokenization, embedding
- Learning:
  - SGD, momentum/Adam, l.r. vs batch size
- One conceptual question about:
  - LoRA
  - VAE vs GAN (even though we didn’t do a weekly quiz on this)

Calculation questions

No “multi-part” calculations

Simpler versions of quiz calculation questions

- Dot product attention
- Contrastive learning
- Calibration error
- Convolution

Quiz 1: transformers

Transformer basics: GPT versus BERT

Tokenization

Temperature

Attention (calculation, and properties like quadratic scaling)

Get some scratch paper ready for the next few.
Suppose we have three words, "Attention to words", and they have word embeddings, $x_{\text{Attention}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, x_{\text{to}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, x_{\text{words}} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. For simplicity, assume that the key, query and value matrices for each word are all equal,
$k_i = \mathbb{I}\, x_i, q_i = \mathbb{I}\, x_i, v_i = \mathbb{I}\, x_i$.
We're going to calculate the output of a dot-product attention on this input. (Don't use scaling, so we get softmax(QK^T) V, where Q, K, V are matrices that stack the vectors, like $Q = \begin{pmatrix} - & \vec{q}_1 & - \\ - & \vec{q}_2 & - \\ - & \vec{q}_3 & - \end{pmatrix}$).
First we need the dot product matrix, then we take the softmax. The output of the second position (with "to" as the input) pays the most "attention" (i.e. has highest probability) to which token?

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	words	19	79%
×	to	3	13%

Temperature

My GPT model says that the probability distribution over my next token is "dog": 70% or "cat": 30%.
If I set the "temperature" very close to zero (equivalent to multiplying the logits in a softmax by a large number), what will be the result?

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	"Dog" will be sampled with probability near 100%	53	96%

Suppose we remove positional embeddings from a transformer. What happens?

oPE = No Positional Embedding – causal attention can still *infer* position by looking at size of context!

https://sebastianraschka.com/llm-architecture-gallery/nope/

Answer Frequency Summary

Answer	Respondents	Percentage
✓ The model can't distinguish different word orders.	50	91%

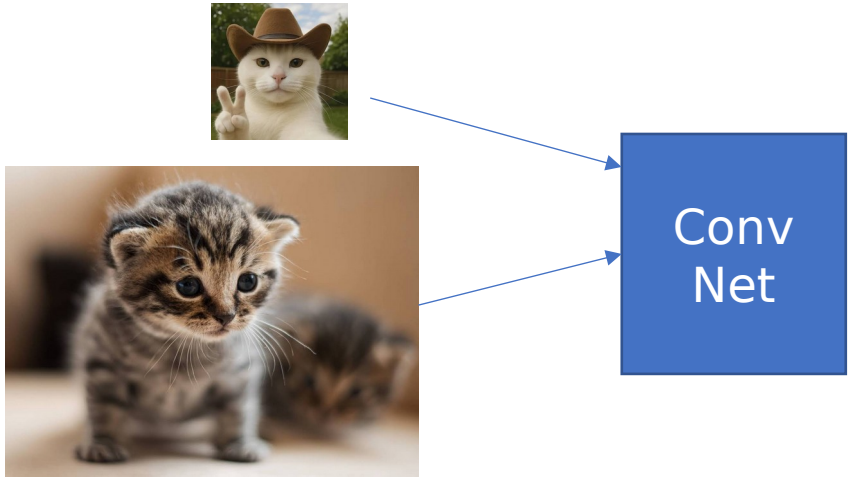
Quiz 2

Vision, convolution

Which elements could NOT appear in a "fully convolutional" neural network?

Pooling	4 respondents	7 %
Dropout	7 respondents	13 %
Strided convolutions	1 respondent	2 %
Fully connected layers	**42 respondents**	**76 %** ✓
"Deconvolution" or "transposed convolutions"	1 respondent	2 %

Fully connected, or linear layers, have to have a fixed input / output dimension size!



Should work on any size image!

1x1 convolution – see demo!

Here's another weird thing you can do with kernels that we didn't discuss in class - apply a kernel with size 1 (in 1-d case, or 1x1 in 2-d case). What would be the meaning of this? Let's consider this in the 1-d case.

Imagine we have a signal, x=[[3],[1],[5],[2]], which we can think of as having length 4 but only one channel. Now we apply a kernel of size 1, with 2 channels. The kernel for channel 1 is $h_1 = 0$, the kernel for channel 2 is $h_2 = 2$, so we might say $h = [[0], [2]]$, What could represent the output?

[[0,6], [0, 2], [0,10], [0,4]]	**45 respondents**	**82 %**	 ✓
[[6],[2],[10],[4]]	**9 respondents**	16 %	
[2, 10, 4]	**1 respondent**	2 %	

A regular convolution operation (1-d)

Given a 1-d input signal $x = [3, 1, 2, 4]$ and a kernel $h = [2, 1]$, calculate the 1-d convolution of x with h (sometimes we denote that as $x \star h$ in signal processing). Assume that the convolution is 'valid', meaning no padding is used. What is the resulting output?

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	[7,4,8]	51	93%

Out of curiosity, how did you solve this?
(93% correct!)

$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ Which matrix is rank one?

$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

Quiz 3

Contrastive and confidence

Suppose that I make the following predictions on each day:
Monday: I predict rain with confidence 80% - it actually rains
Tuesday: I predict rain with confidence 80% - it actually rains
Wednesday: I predict rain with confidence 80% - it actually rains
Thursday: I predict rain with confidence 80% -it actually rains
Friday: I predict rain with confidence 80% - it actually rains

What is my calibration error? Was I over-confident or under-confident?

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	0.2, under-confident	47	90%

For some data, (x,y), my neural network makes the following predictions:

True label	X	Prediction
Y=1	X=1.4	P(Y=1|X=1.4) = 0.8
Y=1	X=2.7	P(Y=1|X=2.7) = 0.8
Y=1	X=3.1	P(Y=1|X=3.1) = 0.8
Y=1	X=4.0	P(Y=1|X=4.0) = 0.8
Y=0	X=4.5	P(Y=1|X=4.5) = 0.2
Y=0	X=6.7	P(Y=1|X=6.7) = 0.2
Y=0	X=6.8	P(Y=1|X=6.8) = 0.2
Y=0	X=6.4	P(Y=1|X=6.4) = 0.2

What's the expected calibration error (ECE)? (Hint: What is the predicted label in each case? What is the *confidence* for this prediction?)

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	0.2	42	81%

We want to train a model to learn good embeddings using a contrastive loss. Consider that we have embeddings for three points: an "anchor point" z_a, a "positive" z_p, and a "negative" z_n. The goal of contrastive losses is to build a loss that moves the positive point closer to the anchor, and the negative point further away.
We will consider the "InfoNCE" or Contrastive Predictive Coding loss style. We turn this into a **2-class classification problem** where the model must choose which candidate (z_p or z_n) matches the anchor. We consider the dot products, $z_a \cdot z_p, z_a \cdot z_n$ as the logits for predicting whether the anchor is closer to the positive or negative point. Use cross entropy to write the training loss for this example in terms the embeddings.

$$-\log \frac{e^{z_a \cdot z_p}}{e^{z_a \cdot z_p} + e^{z_a \cdot z_n}}$$

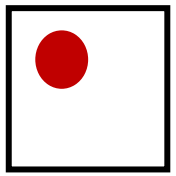
$$-\log \frac{e^{z_a \cdot z_n}}{e^{z_a \cdot z_p} + e^{z_a \cdot z_n}}$$

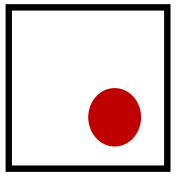
$$-\log \sigma(z_a \cdot z_p) - \log \sigma(-z_a \cdot z_n)$$

$$-\sum_{y=\{p,n\}} q(y) \log p(y)$$

83% correct

Contrastive learning picture

$\vec{z}_a = f($  $) = \begin{bmatrix}1\\0\end{bmatrix}$

$\vec{z}_+ = f($  $) = \begin{bmatrix}0.5\\0\end{bmatrix}$ Y=1

$\vec{z}_- = f($  $) = \begin{bmatrix}0\\-1\end{bmatrix}$ Y=0

$$q(Y=1)=1, q(Y=0)=0$$

The true label (q) is that + is the positive, and – is the negative
Now the contrastive probability based on dot product similarity:

$$p(Y=1|z_+,z_-)=\frac{e^{z_+\cdot z_a}}{e^{z_+\cdot z_a}+e^{z_-\cdot z_a}}$$
$$p(Y=0|z_+,z_-)=\frac{e^{z_-\cdot z_a}}{e^{z_+\cdot z_a}+e^{z_-\cdot z_a}}$$

And the cross entropy will be:

$$H(q,p)=-\sum_{y=0,1} q(y)\log p(y|z_+z_-)$$
$$H(q,p)=-z_+\cdot z_a+\log e^{z_+\cdot z_a}+e^{z_-\cdot z_a}$$

Quiz 4

Learning dynamics, SGD, momentum, Adam

You just finished tuning the hyper-parameters for your neural network trained using SGD on a single GPU:

batch size = 10

dropout = 0.1

learning rate = 0.001

weight decay = 0.01

Now, you're going to do your training in parallel on 4 GPUs, leading to a new batch size = 40.

What other hyper-parameter do you need to change to get comparable results to your single GPU results?

(You can see this calculation appears explicitly in the NanoGPT training code.)

Math about batch size and learning rate

$$\Delta x = g(x)\Delta t + s(x)\epsilon$$
$$\epsilon \sim N(0, \Delta t)$$

Stochastic differential equation:

$$\mathrm{d}x = g(x)\mathrm{d}t + s(x)dW$$

What goes here determines the amount of noise or exploration

For SGD with learning rate, η,

We use CLT to derive

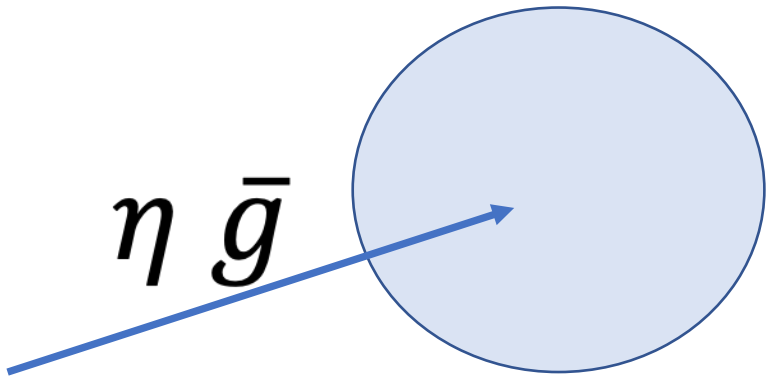
$$\Delta\theta = \bar{g}(\theta)\,\eta + \sqrt{\frac{\eta}{batch\ size}}\,\epsilon$$

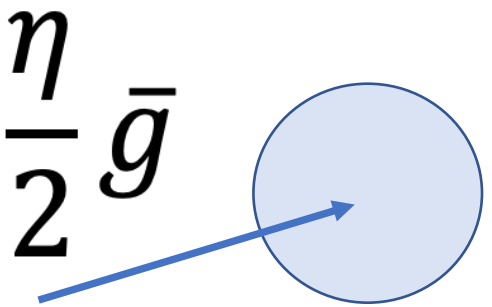
$$\epsilon \sim N(0, \eta)$$

$$\frac{\text{Learning rate}}{\text{Batch size}}$$

Should be kept constant to keep noise/exploration constant

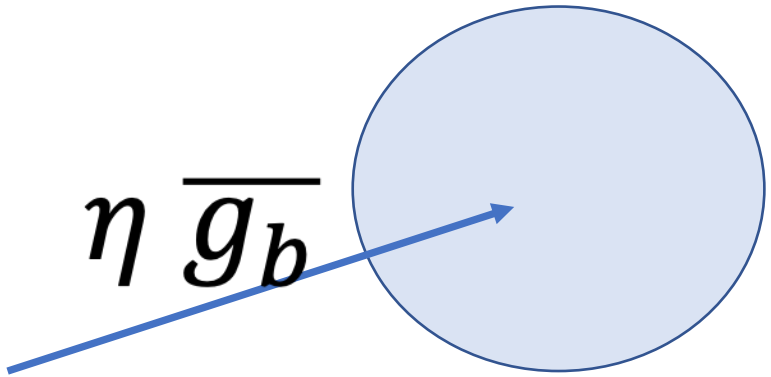
Visual intuition – batch size

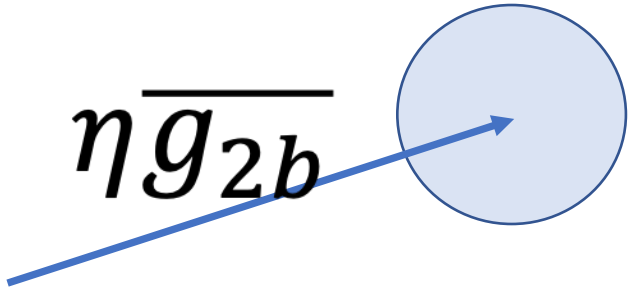




Halve the learning rate, halve the noise

Visual intuition – batch size



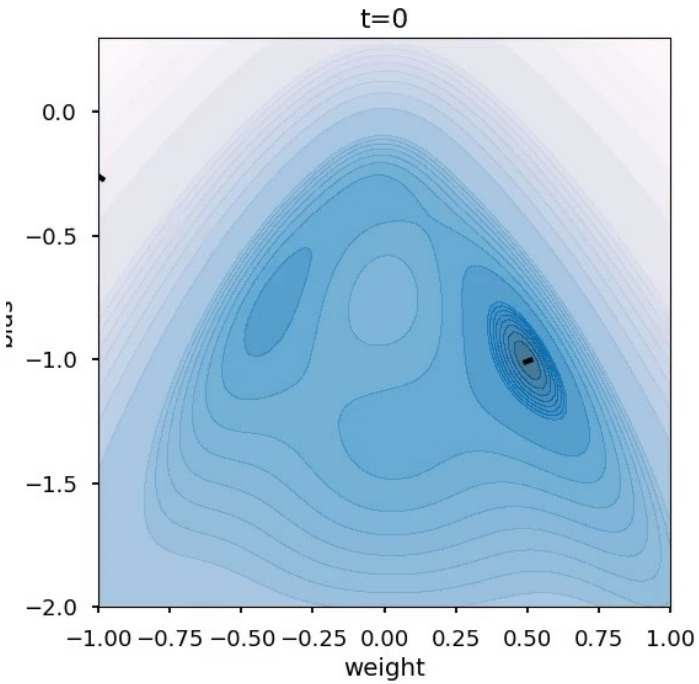


Double the batch size – reduce the noise

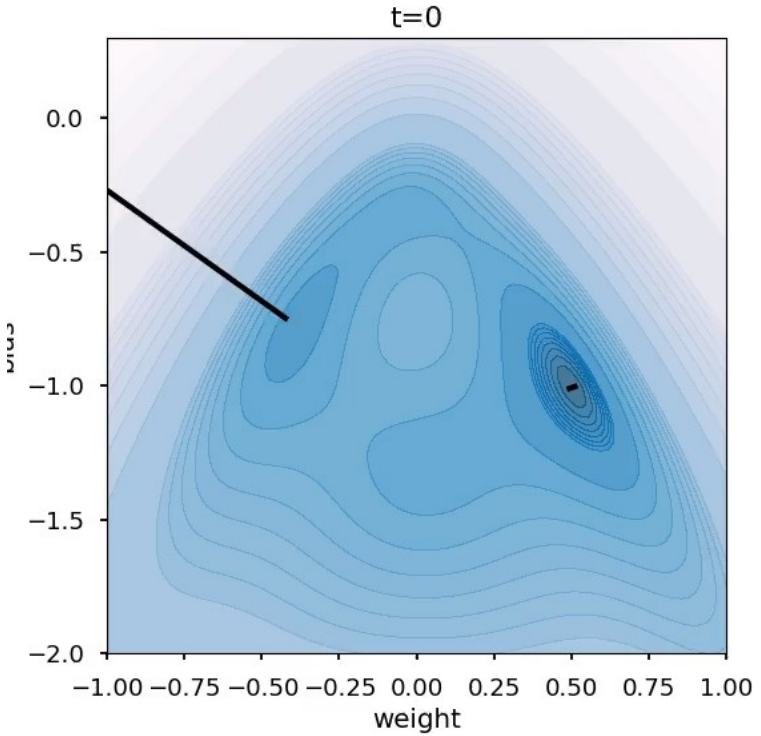
Full batch gradient descent is more likely to get stuck in a local optima than (mini-batch) stochastic gradient descent.

True	98 respondents	83 %	✓
False	20 respondents	17 %	

Small step size (less noise, closer to full batch)



Large step size (noisier)

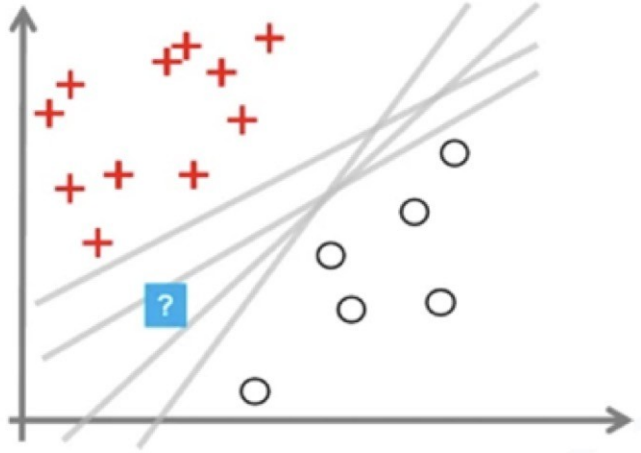


Bayesian model averaging (not on final)

Suppose that you trained four neural networks on the training data with a maximum likelihood objective, leading to models with weights w_1, w_2, w_3, w_4.

You run each model on a test point, labeled "?" in the image, leading to the following predictions.

$$p(y = +|x = ?, w_1) = 0.95$$
$$p(y = +|x = ?, w_2) = 0.45$$
$$p(y = +|x = ?, w_3) = 0.4$$
$$p(y = +|x = ?, w_4) = 0.3$$



If you ensemble the predictions as an approximation of the Bayesian model average, what prediction will you make?

$p_{ensemble}(y = +|x = ?) = 0.25$	5 respondents	4 %
$p_{ensemble}(y = +|x = ?) = 0.525$	101 respondents	86 % ✓
$p_{ensemble}(y = +|x = ?) = 0.5$	12 respondents	10 %

In the last question, we used an ensemble of neural networks to approximate the Bayesian model average. What would have made this a better approximation of the Bayesian model average?

Sample the weights from the Bayesian posterior, rather than optimizing for the maximum likelihood solutions	100 respondents	85 % ✓
Ensemble the arg max predictions, rather than the probabilities	13 respondents	11 %
Use a different prior	5 respondents	4 %

Momentum

Consider a simple scenario where you are using a momentum-based SGD optimizer. If the previous velocity/momentum vector v_{t-1} = [0.9, 0.1] and the current gradient Unknown node type: br g_t is [-0.5, 0.3], with a momentum $\mu = 0.9$ and learning rate $\eta = 0.1$, calculate the update in the velocity vector.

Use the formula for momentum update (scaling follows the Sutskever paper that we showed in class):

$$v_t = \mu v_{t-1} - \eta g_t$$

v_t = [1.4, -0.2]	9 respondents	8 %	
v_t = [0.86, 0.06]	**104 respondents**	**88 %**	✓
v_t = [0.4, 0.4]	5 respondents	4 %	

This class 94% correct, way better at simple algebra!

Following the last question, the next step would be to update the parameters based on the momentum. If we had done gradient descent without momentum, the update would have been $\Delta\theta = -\eta g_t = [0.06, -0.03]$ instead of $\Delta\theta = v_t$ from your previous answer. The second variable would have actually been in the *opposite* direction to the one based on momentum. Conversely, the update we make to the first variable is in the same direction as the previous momentum, so we end up making an even larger step in that direction than we would get from doing gradient descent.

Besides altering the final parameter updates, what's another difference between using gradient descent with and without momentum?

Momentum introduces second order derivatives of the loss function, while gradient descent without momentum uses first order derivatives of the loss.	21 respondents	18 %	
We have to store the momentum/velocities between updates. No storage was required for gradient descent without momentum.	**88 respondents**	**75 %**	✓
Introducing momentum guarantees that we can't get stuck in local minima.	9 respondents	8 %	

Adam

To motivate the Adam optimizer, we interpreted the tracking of momentum as being like an exponential moving average (EMA) of the gradients. Therefore, we can imagine that the gradients are a noisy time-series that we smooth with EMA.

In addition to the EMA of the gradients, what other quantity does Adam track an EMA of?

The variance of the gradients, $(g_t - v_t)^2$, where v_t approximates the current mean.	25 respondents	21 %	
The learning rate	6 respondents	5 %	
The squared gradients, g_t^2	**87 respondents**	**74 %**	✓

90%

We can interpret Adam as adding an adaptive learning rate, because the effective learning rate is the learning rate divided by the EMA of the (uncentered) second moment of the gradients.

In what ways is this learning rate "adaptive"?

(B) The effective learning rate may be different for each variable.	8 respondents	7 %	
(A) and (B)	**99 respondents**	**84 %**	✓
(A) The effective learning rate changes over time.	11 respondents	9 %	

92%

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM

2. Momentum and Nesterov's Accelerated Gradient

The momentum method (Polyak, 1964), which we refer to as classical momentum (CM), is a technique for accelerating gradient descent that accumulates a velocity vector in directions of persistent reduction in the objective across iterations. Given an objective function $f(\theta)$ to be minimized, classical momentum is given by:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

Adam update – exponential moving average

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon},$$

Hyper-parameters:

$$\beta_1 = 0.9$$
$$\beta_2 = 0.999$$
$$\eta = 0.001$$

One of the advantages of Adam is that the defaults work well for many situations. SGD with well-tuned hyper-parameters is often better, but much harder.

Homework / coding questions

I’ll do a 5-10 minute overview Monday based on results

So far, most have done well.

Homework-related questions

- HW 0: MLP, basics (2) Generated name detector
- HW 1: Transformer (2) Alien CalcGPT
- HW 2: Confidence (1) Confident Emoji hunter
- ~~HW 3: Diffusion (0)~~

Week 10: special topics

Let me know if you have particular interests